```
# Install packages needed to build gems
RUN apt-get update -qq && \
    apt-get install --no-install-recommends -y build-essential git \
      libyaml-dev pkg-config && \
    rm -rf /var/lib/apt/lists /var/cache/apt/archives

# Install application gems
COPY Gemfile Gemfile.lock ./
RUN bundle install && \
    rm -rf ~/.bundle/ "${BUNDLE_PATH}"/ruby/*/cache \
      "${BUNDLE_PATH}"/ruby/*/bundler/gems/*/.git && \
    bundle exec bootsnap precompile --gemfile

# Copy application code
COPY . .

# Precompile bootsnap code for faster boot times
RUN bundle exec bootsnap precompile app/ lib/

# Precompiling assets for production without requiring secret
#   RAILS_MASTER_KEY
RUN SECRET_KEY_BASE_DUMMY=1 ./bin/rails assets:precompile
```

This `FROM` instruction specifies that the previous image is to be used as the base for this step. The importance of this will become clear when we get to the final image.

Next, we install some packages that are needed to build the application:

- `build-essential` is a package that installs the basic tools needed to compile software.
- `git` is a version control system.
- `pkg-config` is a helper tool used when compiling software.

Next, is a `COPY` instruction which copies the `Gemfile` and `Gemfile.lock` from your application to the image. This is done separately from the rest of the application code to take advantage of Docker's caching mechanism. If